

# Claude Skills — das Grundkonzept im Überblick

---

Quellen: [video-summary-sQBinJA\\_zxU.md](#), [claude-oekosystem-ueberblick.md](#), [video-summary-B\\_OqkMRFonM.md](#), [video-summary-KWrSLqnB6vA.md](#), [video-summary-TP73qyFWDcY.md](#), [video-summary-rRF3pAEQuzM.md](#), [video-summary-AL391nkWGlc.md](#), [video-summary-wZeOwqmSw84.md](#), [video-summary-HcWpz-0YLRo.md](#), [video-summary-tK9C3Skskws.md](#), [video-summary-8D8ewFBjfFM.md](#), [video-summary-Lu95f1ZBlos.md](#)

Skills tauchen in praktisch jedem zweiten der angeschauten Videos auf, aber immer nur am Rand. Dieser Artikel bündelt das Grundkonzept, das in keiner Einzel-Zusammenfassung vollständig steht (siehe [notes-audit-report.md](#)).

## Was ein Skill technisch ist

---

Ein Skill ist kein simpler Custom-Instruction-Text, sondern ein echter lokaler Ordner mit strukturierten Markdown-Dateien und ggf. kleinen Skripten (sQBinJA\_zxU) — im Kern eine Datei, die festlegt, was Claude tun soll, welche Schritte es befolgen muss und wie das Ergebnis aufgebaut sein soll (rRF3pAEQuzM). Ein kompletter, wiederverwendbarer Arbeitsablauf, der per Triggerwort gestartet wird — laut B\_OqkMRFonM "wahrscheinlich der wichtigste Punkt" im gesamten Funktionsumfang von Claude.

## Wie Skills mit anderen Bauteilen zusammenspielen

---

- **Slash-Commands** sind die Trigger für einen Skill ( /stichwort ) — nicht der Skill selbst (B\_OqkMRFonM)
- **Plugins** bündeln mehrere Skills zu einem installierbaren Paket (B\_OqkMRFonM)
- **Scheduled Tasks** lassen Skills automatisch zu festen Zeiten laufen (B\_OqkMRFonM, wZeOwqmSw84)
- **MCP** ist etwas anderes als ein Skill: Skill = Wissen ("die Fahrstunde"), MCP = der Werkzeugzugriff, der KI-Absichten in konkrete API-Aufrufe übersetzt ("der Autoschlüssel") — Details dazu in [mcp-ueberblick.md](#)

## Das Kontextfenster-Problem und seine Lösung

---

Man kann nicht bei jedem Prompt das komplette "Firmenhandbuch" mitgeben. Die Lösung heißt **Progressive Disclosure**: das System sucht per semantischer Suche im Hintergrund nur die eine passende Skill-Datei heraus und lädt nur diese ins aktive Gedächtnis (sQBinJA\_zxU, claude-oekosystem-ueberblick.md).

## Wie man gute Skills baut

---

- **Nie abstrakt planen, immer aus einer konkreten Konversation heraus** (TP73qyFWDcY): Der Use Case ist bereits validiert, weil man die Arbeit gerade manuell gemacht hat. Prompt-Muster: *"Based on this conversation, build me a skill"*
- **Nach dem Bauen einen "Gotchas"-Abschnitt ergänzen**, der Fehler und Edge Cases dokumentiert, damit sie sich nicht wiederholen (TP73qyFWDcY)
- **Faustregel für Kandidaten**: Ein manueller Schritt, der mehr als 1-2x nervt, ist der Kandidat für einen Skill — nicht eine hypothetische Idee (ai-agent-workflow.md, destilliert aus Peter Steinbergers Workflow)
- **Einstiegs-Prompt, falls unklar was sich lohnt**: *"Based on the project I'm working on, what Claude Skills should I create?"* (KWrSLqnB6vA, Boris Chernys Workflow) — Analogie dort: ein Prompt ist "dribble the ball", ein Skill der "einstudierte Spielzug"

- Erstellt werden Skills über den offiziellen `/skill-creator`, der per Rückfragen durch den Prozess führt (rRF3pAEQuzM)
- **Neuer, einfacherer Weg seit Ende Juli 2026: "Record a Skill" in Claude Cowork** — statt den Ablauf zu beschreiben, zeichnet man ihn per Bildschirmaufnahme (Klicks, Tipptext, Stimme) einmal vor, Claude leitet daraus automatisch den Skill ab. Nur in der Desktop-App (Tab "Cowork"), nicht im normalen Chat. Feature-Existenz per Websuche bestätigt (HcWpz-0YLRo, tK9C3Skskws — zwei unabhängige Demos mit wortgleichem Consent-Dialog: *"Your screen, clicks, typing, and voice are recorded, then sent to Claude and turned into a repeatable skill. Don't type passwords or secrets..."*). Ein drittes, unabhängiges Video ([video-summary-KjNp6CbHUSY.md](#)) bestätigt dasselbe Feature (identischer Consent-Dialog) und ergänzt zwei praktische Tipps: das stärkste verfügbare Modell zum **Bauen** eines Skills nutzen, ein günstigeres zum späteren **Ausführen**; erstellte `.skill`-Dateien lokal sichern statt sich allein auf die Cloud-Kopie zu verlassen (Resilienz-Argument, passt zur "Spannungsfeld"-Empfehlung unten).

## Vier-Bereiche-Modell für Agent-Arbeit (Harness/Context/Tool/Skill Engineering)

Ein Video (tK9C3Skskws) ordnet Skill-Bau als vierten von vier Bausteinen professioneller Agent-Arbeit ein: **Harness Engineering** (die Umgebung/Werkzeuge, in der der Agent läuft), **Context Engineering** (welches Wissen er bekommt), **Tool Engineering** (MCP-Anbindungen) und **Skill Engineering** (wiederholbare Prozesse). Ein zweites, unabhängiges Video (8D8ewFBjFM) verwendet "Harness" enger und beiläufiger als Sammelbegriff für Skills selbst ("ein Skill ist im Kern really just harness"), ein drittes (SFtiPOTLBHA) nutzt ein ganz anderes, chronologisches Stufenschema (Prompt → Context → Harness → Loop → Graph Engineering). Die drei Videos widersprechen sich nicht direkt, benutzen "Harness" aber uneinheitlich — noch keine im Repo aufgelöste Definition, eher ein Hinweis, dass sich die Begriffswelt rund um Agent-Engineering 2026 noch nicht gesetzt hat.

## Modulare vs. monolithische Skill-Architektur (Matt Pocock vs. Superpowers/gstack/GSD)

Neben Anthropic's offiziellem `/skill-creator`-Weg existieren community-gebaute Skill-Sammlungen mit gegensätzlicher Philosophie (8D8ewFBjFM):

- **Monolithisch** (obra/superpowers, garrytan/gstack, gsd-build/get-shit-done): eine feste Kette Brainstorming → Plan → Tickets → Implementierung → Review, die komplett neu durchlaufen werden muss, wenn mittendrin ein Fehler auffällt.
- **Modular** (mattpocock/skills, "Five Bricks, Any Order"): fünf unabhängig triggerbare Skills — `/grill-me` (strukturiertes Interview bis zum gemeinsamen Verständnis, keine Umsetzung ohne Bestätigung), `/to-spec` (Konsens als reine Text-Spec ohne Code, da Code im Spec veraltet), `/to-tickets` (Zuschnitt nach Feature statt technischer Schicht), `/implement` (TDD, "Red First, Always") und `/code-review` (läuft in frischem Kontextfenster, feste Checkliste aus 12 Code-Smells nach Martin Fowlers *Refactoring*).
- **Meta-Prinzip fürs Skill-Schreiben selbst**: radikales Kürzen ("jedes überflüssige Wort lenkt ab") plus Begriffe mit fester, tiefer Bedeutung statt langer Erklärungen ( `/writing-for-agents` ).
- **"Deep Module"/Deletion-Test** als Architekturprinzip: ein Hauptprogramm ruft pro Aufgabe nur eine Funktion auf ("eine Tür"), die intern beliebig viele Unterfunktionen kapselt — spart beim Debuggen Kontextfenster-Tokens. Test, ob eine Funktion nötig ist: löschen und schauen, ob etwas bricht.

Nicht im Repo verifiziert, sondern als persönliches Fazit des Video-Hosts markiert: dass stärkere Modelle (Opus 5/Fable 5) weniger starre Guardrails bräuchten als schwächere, gestützt auf Anthropic's reale ~80%-Kürzung des Claude-Code-Systemprompts für neuere Modelle.

## Beispiele aus den Videos

- **rRF3pAEQuzM** zeigt fünf konkrete Wissensarbeits-Skills im Detail: Präsentationsarchitekt (SCR-Framework), Aufgabenextraktor (5W1H aus Meeting-Transkripten), Qualitätsprüfer (systematisches Output-Review), Kompaktmodus (kürzere Antworten, spart Output-Tokens) und Kontext-Kompressor (verdichtet lange Chats zu einem "Context Pack", spart Input-Tokens)
- **AL391nkWGlc** zeigt einen domänenspezifischen Skill: "home-assistant-best-practices", installierbar via `npx skills add` oder Plugin-Marketplace, mit Entscheidungsworkflows und Namenskonventionen für Smart-Home-Automatisierungen

## Spannungsfeld: Skills als Produktivitätsgewinn vs. Vendor-Lock-in

---

Zwei der Videos ziehen aus Skills gegensätzliche Schlüsse, ohne sich direkt zu widersprechen — es sind einfach unterschiedliche Blickwinkel: [video-summary-6LVB3mpPvB4.md](#) empfiehlt den Aufbau eigener Skills explizit als **Resilienz-Strategie** gegen den Ausfall eines einzelnen KI-Anbieters (siehe [fable-5-modell-sperre.md](#)). [video-summary-sQBinJA\\_zxU.md](#) warnt umgekehrt: Wer hunderte Skills/MCP-Interfaces speziell für einen Anbieter baut, bindet sich gerade dadurch stark an diesen. Auflösung: Die Skill-Definition selbst (Markdown + Beschreibung) ist meist anbieterunabhängig übertragbar — die Bindung entsteht eher durch anbieterspezifische Trigger-Mechanik/Slash-Commands als durch den Skill-Inhalt selbst.

### Konkreter Malware-Beleg für die Marktplatz-Warnung (ClawHub)

Was bis hierhin eine eher allgemeine Warnung war, ist durch [video-summary-Lu95f1ZBlos.md](#) mit einem harten, unabhängig verifizierten Einzelfall unterlegt: Auf ClawHub wurde der meisheruntergeladene Skill dort — ein "What Would Elon Do?"-Imitator — mit versteckter Datenexfiltration als Malware identifiziert. Per WebSearch bestätigt (The Hacker News, Unit42/Palo Alto, PointGuard AI): 230-341 schädliche Skills seit Ende Januar 2026 auf ClawHub gefunden. Zusätzlich fand eine Snyk-Studie ("ToxicSkills") Prompt-Injections in 36% der geprüften Skills auf ClawHub/skills.sh, wobei Trail-of-Bits-Forscher demnach alle öffentlichen Skill-Scanner umgehen konnten. Praktische Konsequenz für eine Team-Policy: Skills nur aus der offiziellen Anthropic-Quelle beziehen, nie ungeprüft von Community-Marktplätzen — auch nicht von seriös wirkenden Anbietern wie skills.sh (Vercel-Hintergrund schützt laut Studie nicht vor Prompt-Injection-Skills).

---

## Kernbotschaft

---

Skills sind Claudes wichtigster Baustein für wiederholbare Arbeitsabläufe: echte Ordner mit Markdown-Anweisungen, per Progressive Disclosure kontextfenster-schonend geladen, am besten aus einer bereits geführten Konversation heraus gebaut statt abstrakt geplant. Sie lösen ein reales Produktivitätsproblem, sollten aber nicht blind auf einen einzigen Anbieter zugeschnitten werden.

## Themen-Tags

---

Claude Skills, Skill-Creator, Progressive Disclosure, Slash-Commands, Plugins, Produktivität, Vendor-Lock-in, Record a Skill, Claude Cowork, Harness Engineering, Matt Pocock, Superpowers, gstack, GSD, Deep Module, Code Review